

Application probabilité pour la détection de spam

PMI (Pointwise mutual
information)



Le concept de PMI aide à identifier les mots qui sont interconnectés. En termes simples, il élucide la probabilité que deux mots apparaissent ensemble plus fréquemment que ce qui serait prévu par hasard.

Lorsque les mots "Data" et "Science" sont combinés, ils forment le terme spécifique "Data science", qui porte une signification distincte. Cependant, lorsqu'ils sont utilisés séparément, ces mots ont une signification indépendante. Un cas similaire peut être vu avec l'expression "Great Britan", où le mot "Great" n'a de signification que lorsqu'il est associé à "Great Britan" et non avec d'autres mots comme "UK", "London" ou "Dubai".


$$\text{PMI}(\text{word}_1, \text{word}_2) = \log_2 \left(\frac{P(\text{word}_1 \& \text{word}_2)}{P(\text{word}_1)P(\text{word}_2)} \right)$$

$$P(w) = \frac{\text{Freq}(w)}{\text{totalWordCount}}$$



Dans le cas où 'word1' et 'word2' sont considérés comme non liés, leur probabilité combinée est égale à la multiplication de leurs probabilités individuelles. Si la formule PMI mentionnée ci-dessus donne une valeur de 0, elle indique que le numérateur et le dénominateur sont égaux et que le logarithme de 1 donne 0. En termes simples, cela implique que les mots 'word1' et 'word2' n'ont pas de signification ou de pertinence particulière lorsqu'ils sont utilisés ensemble.



Pour déterminer la fréquence d'un mot spécifique, on peut simplement compter le nombre de fois qu'il apparaît. De même, la probabilité que deux mots apparaissent ensemble peut être évaluée en comptant les cas dans lesquels ils se produisent dans une certaine proximité les uns des autres.

Calcul de la somme des valeurs de PMI pour un document :

Soit D un document contenant une liste de mots $\{w_1, w_2, \dots, w_n\}$ et une liste correspondante de contextes $\{c_1, c_2, \dots, c_n\}$.

La somme des valeurs de PMI pour le document D est donnée par :

$$PMI_{\text{global}}(D) = \sum_{i=1}^n PMI(w_i, c_i)$$

Probability a respondent mentions "church" and is *religious*

The word "church"

$$\text{PMI}(w, c) = \log \frac{p(w, c)}{p(w) p(c)}$$

The category *religious*

Probability of any respondent mentioning "church"

Probability of any respondent being *religious*

$$ScorePositif(w) = PMI(w, \{positif\}) - PMI(w, \{negatif\} \cup \{neutre\})$$

$$ScoreNegatif(w) = PMI(w, \{negatif\}) - PMI(w, \{positif\} \cup \{neutre\})$$

$$ScoreNeutre(w) = PMI(w, \{neutre\}) - PMI(w, \{positif\} \cup \{negatif\})$$

$$PMI(w, \{NomDeLaClasse\}) = \log_2 \frac{freq(w, \{NomDeLaClasse\}) * N}{freq(w) * freq(\{NomDeLaClasse\})}$$

$freq(w, \{NomDeLaClasse\})$ est le nombre de fois où le mot w apparaît dans un tweet appartenant à $\{NomDeLaClasse\}$, $freq(w)$ est le nombre d'apparitions du mot w dans le corpus, $freq(\{NomDeLaClasse\})$ est le nombre de tweets appartenant à la classe $NomDeLaClasse$ et N est le nombre total de termes dans le corpus.


```
data = {  
    'text': [  
        'Free money now',  
        'Call now to claim your prize',  
        'Meet me at the park',  
        'Let's catch up later',  
        'Win a new car today!',  
        'Lunch plans?',  
        'Congratulations! You won a lottery',  
        'Can you send me the report?',  
        'Exclusive offer for you',  
        'Are you coming to the meeting?'  
    ],
```

```
'label': ['spam', 'spam', 'not spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam']
```

Implémentation du PMI dans la détection du sentiment

```
import pandas as pd
# Charger l'ensemble de données
file_path = 'imdb_labelled.txt'
data = pd.read_csv(file_path, delimiter='\t', header=None)
data.columns = ['Review', 'Sentiment']
# Afficher les premières lignes
data.head()
```

```
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk

# Télécharger Les fichiers de données NLTK nécessaires
nltk.download('punkt')
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))
def preprocess_text(text):
    # Supprimer la ponctuation
    text = re.sub(r'[\W\s]', '', text)
    # Tokeniser et convertir en minuscules
    words = word_tokenize(text.lower())
    # Supprimer Les stop words
    words = [word for word in words if word not in stop_words]
    return words

# Appliquer Le prétraitement aux avis
data['Processed_Review'] = data['Review'].apply(preprocess_text)
# Afficher Les premiers avis traités
data.head()
```



```
from collections import defaultdict

# Initialiser les dictionnaires de fréquence
word_freq = defaultdict(int)
sentiment_freq = defaultdict(int)
word_sentiment_freq = defaultdict(lambda: defaultdict(int))

total_docs = len(data)

# Calculer les fréquences
for _, row in data.iterrows():
    words = row['Processed_Review']
    sentiment = row['Sentiment']

    sentiment_freq[sentiment] += 1
    for word in words:
        word_freq[word] += 1
        word_sentiment_freq[word][sentiment] += 1
```

```
import math
```

```
pmi = defaultdict(lambda: defaultdict(float))
```

```
for word, sentiments in word_sentiment_freq.items():
```

```
    for sentiment, count in sentiments.items():
```

```
        p_word = word_freq[word] / total_docs
```

```
        p_sentiment = sentiment_freq[sentiment] / total_docs
```

```
        p_word_sentiment = count / total_docs
```

```
        pmi[word][sentiment] = math.log2(p_word_sentiment / (p_word * p_sentiment))
```



```
def analyze_sentiment(review):  
    words = preprocess_text(review)  
    pos_score = sum(pmi[word][1] for word in words if word in pmi)  
    neg_score = sum(pmi[word][0] for word in words if word in pmi)  
    return 1 if pos_score > neg_score else 0
```

- 
- ▶ IMDB dataset having 50K movie reviews for natural language processing or Text analytics.

This is a dataset for binary sentiment classification containing substantially more data than previous benchmark datasets. We provide a set of 25,000 highly polar movie reviews for training and 25,000 for testing. So, predict the number of positive and negative reviews using either classification or deep learning algorithms.



	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

